Layered Attacks and Defenses for Autonomous LLM Agents: A Taxonomy, Empirical Evaluation, and Defense-in-Depth Architecture

Khalil Abderrazak
École Nationale Supérieure d'Arts et Métiers (ENSAM)
Meknès, Maroc

Hajji Mohamed
École Nationale Supérieure d'Arts et Métiers (ENSAM)
Meknès, Maroc

Abstract*—Autonomous LLM-based agents that reason over multi-step plans, invoke external tools, and ingest data from untrusted sources expose an attack surface qualitatively different from that of stand-alone language models. Single-turn prompt injection, the dominant threat model for isolated LLMs, does not account for the perception, intention, planning, and action stages through which an autonomous agent transforms raw input into real-world side effects. We present a four-layer attack taxonomy that maps adversarial techniques to the specific architectural component they target: the *Perception Layer* (indirect prompt injection), the *Intention Layer* (goal hijacking), the *Planning Layer* (agentic denial of service via token amplification), and the *Action Layer* (unauthorized tool invocation). We instantiate this taxonomy on an HR document-screening agent, evaluating representative attacks across three models (Llama-3.1-8B, Mistral-Small, Llama-3.3-70B) and three defense baselines totaling $2{,}160$ attack trials. We show that regex-based guardrails achieve a 100% false-block rate on legitimate hard-negative inputs while providing zero additional protection against semantically paraphrased payloads (70% ASR under obfuscation, identical to the undefended baseline). To address these failures, we design DEFENDAGENT, a fail-closed, four-layer defense-in-depth wrapper whose Observation Integrity Monitor blocks 63 of 64 obfuscated injection attempts, reducing the attack success rate from 55.2% to 1.6% (95% Wilson CI: $[0.28\%, 8.3\%]$), a 53.6 percentage-point reduction.

I. INTRODUCTION

Stand-alone large language models process a single user prompt and return a completion. Their primary adversarial concern is *direct* prompt injection: a user crafts input that circumvents alignment safeguards within one turn. This threat model, while well-studied, breaks down when the LLM is embedded inside an autonomous agent.

An autonomous agent operates a multi-step reasoning loop—typically a Thought→Action→Observation cycle—that reads documents, queries databases, invokes APIs, and writes back to persistent memory. Instructions no longer originate solely from the user: they arrive inside CVs, emails, web pages, and tool outputs, none of which can be assumed trustworthy. The model must simultaneously interpret data and follow instructions, yet possesses no architectural mechanism to distinguish between the two. This conflation turns every external data source into a potential instruction channel.

Three properties of autonomous agents compound the problem. First, *state persistence*: the agent carries context across steps, so an injection at step k contaminates all subsequent decisions. Second, *tool access*: the agent can execute irreversible side effects (send emails, write files, invoke APIs), converting a corrupted decision into real-world damage. Third, *cost amplification*: a multi-step reasoning loop multiplies the token footprint of a single malicious input, enabling denial-of-service attacks that are structurally impossible against single-shot models.

These observations motivate three research questions:

RQ1 How vulnerable are LLM agents to attacks that target distinct stages of the agentic pipeline—perception, intention, planning, and action?

RQ2 Do conventional defenses (pattern-matching guardrails, semantic filters) generalize to adaptive adversaries who paraphrase and obfuscate their payloads?

RQ3 Can a layered, state-aware defense architecture substantially reduce attack success rates without modifying the underlying agent?

Contributions.

1) A four-layer attack taxonomy that organizes adversarial techniques by the agent component they target, rather than by surface-level attack mechanism.
2) A controlled empirical evaluation spanning three models, four attack families, three defense baselines, and $2{,}160$ live attack trials, with 95% Wilson score confidence intervals on all reported proportions.
3) An adaptive threat corpus of semantically obfuscated payloads that empirically demonstrates the structural failure of regex-based guardrails (0 percentage-point improvement over the undefended agent).
4) DEFENDAGENT, a non-invasive, fail-closed defense wrapper that reduces obfuscated-injection ASR from 55.2% to 1.6% through observation-integrity monitoring, tool sandboxing, and bounded execution.

II. Background

Prompt injection against language models was characterized early by Perez and Ribeiro, who demonstrated that adversarial suffixes and role-play preambles reliably subvert alignment-trained models. Greshake et al. extended the attack to *indirect* prompt injection, showing that malicious instructions embedded in retrieved documents are executed by retrieval-augmented generation pipelines. These results establish that the data–instruction conflation is not merely theoretical.

The transition from retrieval pipelines to autonomous agents widens the attack surface further. Zhan et al. and Ruan et al. showed that agents with tool access can be steered into executing unauthorized actions, while Xi et al. surveyed the broader risks of LLM-based agents, including goal hijacking and planning-loop manipulation. Yang et al. examined how multi-step reasoning amplifies single-turn vulnerabilities. On the defensive side, prior work has explored input sanitization, output filtering, and prompt-level guardrails, but few studies measure these defenses against adaptive adversaries who deliberately evade detection patterns.

Our work differs in two respects. First, we organize attacks by the *architectural layer* they target—perception, intention, planning, action—rather than by the specific prompt-engineering trick employed. Second, we explicitly evaluate defenses under adaptive threat conditions (semantically paraphrased payloads that bypass exact-match filters), producing empirical evidence that static guardrails fail and that state-aware, composable defenses are necessary.

III. Research Design and Threat Taxonomy

A. Problem Statement

LLMs and LLM-based agents do not share the same security properties. A stand-alone LLM receives instructions from a single source: the user prompt. Most attacks against it are variations of direct injection, jailbreaks, or instruction manipulation performed explicitly by the user. An autonomous agent, by contrast, operates within a richer environment—external data sources, memory systems, planning modules, and execution tools—and receives information from multiple origins whose trustworthiness cannot be guaranteed. Malicious instructions may be concealed inside documents, websites, databases, or tool outputs and can influence the agent without direct user involvement.

The research question follows: *How vulnerable are LLM-based agents to attacks targeting different layers of the agentic pipeline, and how effective are common defensive mechanisms at mitigating these threats?*

B. Target Environment

The experimental target is an AI-powered HR screening agent. The agent receives candidate CVs and is responsible for reading documents, extracting relevant information, producing candidate summaries, and issuing recruitment recommendations (Accept, Reject, or Review).

This scenario was selected for three reasons. **(1) Realism:** HR screening is among the most common enterprise use cases for LLM agents. **(2) High exposure to untrusted data:** the agent continuously processes documents supplied by external individuals; attackers naturally control part of the agent's input space. **(3) Experimental simplicity:** the scenario is realistic enough to exercise all four taxonomy layers while remaining amenable to controlled experimentation and reproducible measurement.

C. Layered Attack Taxonomy

Rather than classifying attacks by individual technique, we organize threats by the architectural component they target. The taxonomy contains four layers, each representing a distinct stage in the agent's decision-making pipeline.

1) Layer 1: Perception: The perception layer encompasses the mechanisms through which the agent acquires information from its environment. The attacker's objective is to manipulate what the agent sees, reads, or interprets. Representative attacks include *indirect prompt injection* (malicious instructions embedded in CVs, PDFs, or web pages), *adversarial data corruption* (altering factual content to bias conclusions without issuing explicit commands), and *multimodal injection* (instructions hidden in images or OCR streams).

2) Layer 2: Intention: The intention layer governs the objectives pursued by the agent. Attacks at this level seek to alter the agent's mission rather than its perception. Representative attacks include *goal hijacking* (replacing the original objective with an attacker-controlled one), *instruction overriding* (invalidating prior instructions through explicit override commands), and *payload splitting* (distributing the malicious objective across multiple inputs for reconstruction during reasoning).

3) Layer 3: Planning: The planning layer corresponds to the agent's internal reasoning loop. Modern agents implement variants of *Thought→Action→Observation* cycles. Attacks at this level target the planning process itself: *agentic denial of service* (creating reasoning loops that consume excessive resources), *recursive expansion* (forcing increasingly complex plans until the context window saturates), and *reasoning manipulation* (influencing intermediate reasoning steps).

4) Layer 4: Action and Privilege: The action layer corresponds to tool usage and external operations. Attacks target the agent's ability to execute operations in the environment: *unauthorized tool invocation* (inducing use of tools unrelated to the assigned task), *confused deputy* (abusing the agent's own privileges on behalf of the attacker), and *side-channel exfiltration* (leaking sensitive information through legitimate channels).

D. Selection of Representative Attacks

Evaluating every known attack is infeasible. We select one representative attack per layer, maximizing coverage, reproducibility, experimental clarity, and practical relevance.

Perception: Indirect Prompt Injection. This attack exploits the fundamental inability of language models to distinguish data from instructions. It is highly realistic in HR scenarios where CVs originate from untrusted individuals. Multimodal injection introduces variables outside our scope;

adversarial data corruption primarily affects accuracy rather than agent control.

Intention: Goal Hijacking. This attack measures whether an attacker can replace the agent's intended objective with an adversarial one. Instruction overriding is a mechanism that enables goal hijacking rather than a distinct outcome; payload splitting is harder to evaluate objectively.

Planning: Agentic Denial of Service. This attack produces measurable effects—execution time, iteration count, token consumption—and is experimentally reproducible. Reasoning manipulation is difficult to quantify; recursive expansion is a specialized form of agentic DoS.

Action: Unauthorized Tool Invocation. This attack directly evaluates whether the agent can be induced to execute an unintended action. Success is measured unambiguously through tool invocation logs. Confused-deputy scenarios require more complex privilege architectures; side-channel attacks require additional infrastructure beyond our scope.

E. Threat Model

For each attack, we specify three dimensions. *Attacker capabilities:* what the attacker controls (e.g., a submitted CV). *Attack objective:* what the attacker attempts to achieve (e.g., influencing the agent's recommendation). *Attack surface:* which system component is targeted (e.g., the document-ingestion pipeline). In all cases, the attacker has no access to the model weights, system prompt, or internal state; the attack surface is limited to the content of submitted documents.

F. Defense Baselines

Three agent configurations are evaluated:

Baseline A (Unprotected): A standard ReAct agent with no defensive mechanisms. Measures intrinsic vulnerability.

Baseline B (Guardrailed): Baseline A plus rule-based security constraints—regex pattern matching against a dictionary of 25 known attack signatures, operating in reject mode. Measures the effectiveness of conventional static defenses.

Baseline C (Semantic Isolation): Baseline B plus a two-stage semantic filter—an LLM-based security classifier labels CVs as SAFE or SUSPICIOUS before the HR agent processes them. Measures the effectiveness of intent-level filtering.

G. Evaluation Metrics

The primary metric is the **Attack Success Rate (ASR)**: the fraction of attack trials in which the agent performs the adversarial objective. Secondary metrics include latency overhead, token consumption, tool misuse rate, and resource exhaustion ratio (for DoS scenarios). All proportions are reported with 95% Wilson score confidence intervals to quantify sampling uncertainty.

IV. Empirical Evaluation: Failure of Static Defenses

We evaluate the four representative attacks across three models and three baselines. Phase 1 uses a single-prompt pipeline totaling 2,160 attack evaluations and 540 clean-CV evaluations, all live API calls at temperature $T = 0$. Phase 2 transitions to an autonomous ReAct agent and introduces an adaptive threat corpus.

TABLE I
ATTACK SUCCESS RATE (%) UNDER BASELINE A (NO DEFENSE), WITH 95% WILSON CIS. EACH CELL: $n = 60$ TRIALS (20 VARIANTS $\times$ 3 RUNS).

Model	Inject	Hijack	DoS	Tool
Llama-3.1-8B	80.0 ± 10.0	61.7 ± 11.9	0.0 ± 3.0	5.0 ± 6.0
Mistral-Small	50.0 ± 12.3	51.7 ± 12.3	0.0 ± 3.0	0.0 ± 3.0
Llama-3.3-70B	80.0 ± 10.0	68.3 ± 11.5	25.0 ± 10.7	0.0 ± 3.0

TABLE II
INJECTION ASR (%) ACROSS BASELINES AND FALSE-BLOCK RATE (FBR) ON HARD-NEGATIVE CVS, 95% WILSON CIS. HARD NEGATIVES ARE LEGITIMATE CVS CONTAINING TRIGGER-ADJACENT VOCABULARY.

Model	ASR: A $\rightarrow$ B $\rightarrow$ C	FBR (B)	FBR (C)
Llama-3.1-8B	$80.0 \rightarrow 50.0 \rightarrow 15.0$	100.0 ± 5.7	20.0 ± 13.9
Mistral-Small	$50.0 \rightarrow 35.0 \rightarrow 5.0$	100.0 ± 5.7	3.3 ± 8.0
Llama-3.3-70B	$80.0 \rightarrow 58.3 \rightarrow 0.0$	100.0 ± 5.7	36.7 ± 16.3

A. Phase 1: Baseline Vulnerability (Baseline A)

Table I reports cross-model attack success rates under no defense.

Perception-layer and intention-layer attacks succeed at high rates across all models: indirect injection reaches 80% on both Llama variants, and goal hijacking ranges from 51.7% to 68.3%. The largest model (Llama-3.3-70B, 70B parameters) is the *most* vulnerable—capability does not imply robustness. Action-layer attacks show low baseline ASR (0%–5%), indicating that the agent does not trivially follow tool-invocation instructions under literal payloads.

B. Phase 1: Defense Tradeoffs (Baselines B and C)

Table II shows how defenses affect injection ASR alongside their false-block rate on legitimate hard-negative CVs.

Two findings emerge. First, Baseline B (regex guardrails) blocks 100% of legitimate hard-negative CVs across all three models—a total false-positive rate that renders the defense operationally unusable. Regex patterns trigger on surface vocabulary (e.g., "system override", "email campaign") regardless of semantic context. Second, Baseline C (semantic filter) provides genuine ASR reduction ($80\% \rightarrow 0\%$ on Llama-3.3-70B) but its false-block rate is volatile and model-dependent (3.3% to 36.7%).

C. Phase 2: Adaptive Adversaries Defeat Static Guardrails

Phase 1 used literal attack payloads. A determined adversary would paraphrase. Phase 2 introduces 10 semantically obfuscated payloads—constructed via semantic paraphrasing, leetspeak substitution, and payload fragmentation—that preserve adversarial intent while altering surface form. Offline verification confirms that all 10 payloads bypass every regex in the guardrail dictionary. The agent architecture transitions to an autonomous ReAct loop with ground-truth tool execution logging.

TABLE III
OBFUSCATED INJECTION ASR (%) UNDER PHASE 2 REACT AGENT (GROQ LLAMA-3.1-8B). THE OBFUSCATED CORPUS IS DESIGNED TO BYPASS BASELINE B'S REGEX PATTERNS.

Baseline	Obfuscated ASR	Δ from A
A (No defense)	70.0%	—
B (Regex guardrails)	70.0%	+0 pp
C (Semantic filter)	32.1%	−38 pp

Table III shows the result on Groq Llama-3.1-8B:

Baseline B provides *zero* additional security against the obfuscated corpus. The 70% ASR is identical to the undefended agent. Baseline C reduces ASR by 38 percentage points, confirming that intent-level classification generalizes where lexical matching does not.

D. Agentic Denial of Service: Token Amplification

Agentic DoS differs from conventional resource exhaustion. In a single-shot model, one prompt produces one completion. In a ReAct agent with a step budget of S and per-step output ceiling of T tokens, a single malicious input can trigger up to S sequential LLM invocations, producing an output bounded by $S \cdot T$ tokens. We define the *token amplification factor* as the ratio of total generated tokens to attacker-supplied input tokens:

$$\alpha = \frac{\sum_{i=1}^{S} t_i}{|x_{\text{attack}}|} \tag{1}$$

where t_i is the token count of the i-th step's output and $|x_{\text{attack}}|$ is the token length of the attacker's input payload.

During Phase 2 evaluation, DoS payloads routed through the ReAct loop (up to `REACT_MAX_STEPS` $= 6$ LLM calls per input) repeatedly exhausted the provider's 500K tokens/day quota, forcing rotation across multiple API keys. Under Baseline A, the DoS ASR reached 95.0% (19/20, 95% Wilson CI: $[77\%, 99\%]$). Baseline B reduced this to 14.3% (1/7, CI: $[2.6\%, 51.3\%]$)—an 81 percentage-point drop—because the DoS corpus relies on structural triggers (repetition directives, oversized-output requests) that the regex patterns still match. The confidence intervals do not overlap, establishing directional significance despite the small sample ($n = 7$ for Baseline B DoS; evaluation was halted by design due to quota exhaustion).

This finding sharpens a key insight: *no single defense layer generalizes across attack classes.* Baseline B is useless against paraphrased injection (which evades pattern matching by construction) yet substantially blunts DoS (which relies on structural triggers). Even where the content filter catches the trigger, it does not bound the *cost* once generation begins—token-bounding must operate alongside content filtering, not behind it.

V. DEFENDAGENT: DEFENSE-IN-DEPTH ARCHITECTURE

The empirical results of Sections 4.2–4.4 demonstrate that no single defense mechanism addresses all four attack surfaces. Regex guardrails fail against paraphrased attacks while producing unacceptable false-positive rates. Semantic filters reduce injection ASR but do not bound token consumption. Neither addresses tool-level containment. These observations motivate DEFENDAGENT, a fail-closed, non-invasive defense wrapper that composes four layers into a defense-in-depth architecture around the existing ReAct agent.

A. Design Principles

DEFENDAGENT does not rewrite or duplicate the ReAct reasoning loop. It attaches through two opt-in extension points added to the agent's `evaluate_cv` function—both default to `None`, leaving the baseline code path byte-for-byte unchanged:

- `step_hook`: a per-step callback (Layers 1 and 2) invoked after each parsed model turn, before any tool execution.
- `executor`: a `SandboxedToolExecutor` (Layers 3 and 4) that intercepts every tool call before it reaches the execution layer.

The security model is *fail-closed*: Layers 1–3 raise exceptions that abort the run and return a sanitized BLOCKED result. The attacker's payload is never echoed into downstream success detectors.

B. Layer 1: Anomaly and Token Bounding

Security objective: Bound the computational cost of any single evaluation, eliminating agentic DoS and reasoning explosions.

Detection logic: Three hard limits enforced per reasoning step: (a) a step cap (`MAX_DEFEND_STEPS` $= 3$), (b) a per-turn output ceiling (`MAX_OUTPUT_TOKENS` $= 250$ words), and (c) repeated-cycle detection (identical step text triggers immediate abort).

Assumptions: Legitimate CV evaluations complete in ≤ 3 steps. The step cap is deliberately tighter than the ReAct agent's own budget (`REACT_MAX_STEPS` $= 6$): an agent requiring more than a few steps on a document-screening task is treated as anomalous.

Limitations: Layer 1 does not inspect payload content. It reduces the *ceiling* on token amplification—worst-case output is bounded at $3 \times 250 = 750$ tokens versus 6,000+ in the undefended agent—but cannot prevent attacks that complete within the step budget.

C. Layer 2: Observation Integrity Monitor

Security objective: Detect and block *forged observations*—model-authored text that mimics the outcome of a tool execution (specifically, fabricating evidence that an email was sent).

Detection logic: After each model turn, Layer 2 scans for lines matching the pattern `Observation: <text>` where `<text>` contains any of 11 phrases indicative of a forged email-send outcome (e.g., "email sent", "sent successfully", "email delivered"). Legitimate observations are produced exclusively by the tool executor; a model that writes its own observation line containing these phrases is fabricating tool-execution evidence.

The monitor also maintains a tamper-evident transcript log. Each step is folded into a SHA-256 hash chain seeded with the system prompt:

$$h_i = \mathrm{SHA256}(h_{i-1} \,\|\, \mathrm{strip}(s_i)) \tag{2}$$

where s_i is the text of step i and $h_0 = \mathrm{SHA256}(\text{system\_prompt})$.

Assumptions: The attacker's strategy involves forging an observation to convince the agent (or a downstream auditor) that an unauthorized action completed. This assumption is empirically validated: 63 of 64 obfuscated attacks in Phase 3 produced exactly this pattern.

Limitations: Layer 2 does not flag benign content observations or legitimate `save_evaluation` echoes. It specifically targets the provenance-fabrication attack vector. An attacker who achieves decision manipulation through the legitimate `save_evaluation` tool—without forging any observation—escapes Layer 2 detection entirely.

D. Layer 3: Instruction Firewall

Security objective: Restrict the agent's tool usage to a whitelist of legitimate HR operations and block known injection signatures in tool action fields.

Detection logic: (a) Tool allowlist: only `save_evaluation` is permitted; any other tool name triggers a hard block. (b) Injection signature scanner: 25 literal substrings (e.g., "ignore previous", "system prompt", "jailbreak", "base64") are matched against the concatenation of tool name and tool input. A match triggers a hard block.

Assumptions: Legitimate HR agents need only one tool to complete their task. Any attempt to invoke an unauthorized tool indicates either a compromised decision or a model hallucination.

Limitations: Layer 3 effectively blocks naive attackers who use literal injection phrases or attempt to invoke undefined tools. It does *not* reliably stop heavily obfuscated attacks: the Phase 2 obfuscated corpus bypasses literal matching by construction. Layer 3's contribution against adaptive adversaries is limited to catching edge cases (e.g., model hallucinations that invoke undefined tools).

E. Layer 4: Capability Sandbox

Security objective: Prevent execution of the restricted tool (`send_email`) under all circumstances, regardless of model intent.

Detection logic: The `SandboxedToolExecutor` intercepts any call to `send_email` before execution. It returns a failure observation ("`SandboxSecurityException: send_email is disabled`") so the model believes the tool ran and failed. The ground-truth `tool_invoked` flag remains `False`; the infrastructure is never touched.

Design rationale: Layer 4 is deliberately *deceptive*. The tool remains visible in the agent's tool specification (the model can still reason about it and attempt to call it), but execution is silently neutralized. This prevents the attacker from observing the capability restriction and adapting.

TABLE IV
ATTACK SUCCESS RATE: PHASE 2 (REACT) VS. PHASE 3 (DEFENDAGENT), GROQ LLAMA-3.1-8B. WILSON 95% CIS REPORTED FOR EMPIRICALLY MEASURED CELLS.

Attack Family	P2 ASR	P3 ASR	95% Wilson CIs	Layer
Obfusc. Injection	55.2%	**1.6%**	[45.3, 64.8] vs. [0.28, 8.3]	L2
Tool Invocation	49.1%	**0%**	architectural†	L4
Agentic DoS	74.1%	**bounded**	architectural†	L1

†Architectural guarantee; not empirically re-measured in Phase 3.

Assumptions: The restricted tool set is known at deployment time. Layer 4 provides an *architectural guarantee*: no code path permits `send_email` execution.

F. Composition

The four layers compose as a defense-in-depth stack. Layer 1 bounds the attack's computational reach. Layer 2 catches the dominant obfuscated-injection strategy (observation forging). Layer 3 filters naive attacks and model hallucinations at the tool level. Layer 4 provides a last-resort architectural guarantee against unauthorized tool execution. No single layer is sufficient; each addresses a distinct attack vector that the others miss.

VI. RESULTS AND LIMITATIONS

A. Phase 3 Evaluation Setup

DEFENDAGENT was evaluated on the same obfuscated injection corpus used in Phase 2, with the same model (Groq Llama-3.1-8B). The Phase 3 evaluation dataset consists of $n = 64$ non-error obfuscated attack trials.

B. Main Result

Table IV compares Phase 2 (ReAct agent, no DEFENDAGENT wrapper) and Phase 3 (ReAct agent with DEFENDAGENT) attack success rates.

The obfuscated injection ASR drops from 55.2% (53/96) to 1.6% (1/64), a 53.6 percentage-point reduction. The 95% Wilson confidence intervals—[45.3%, 64.8%] for Phase 2 and [0.28%, 8.3%] for Phase 3—do not overlap, establishing statistical significance.

C. Layer Attribution

Of the 64 Phase 3 trials:

- **Layer 2 (Observation Integrity Monitor)** blocked 63 of 64 obfuscated injection attempts (98.4%). In every blocked case, the attack caused the model to write a forged `Observation:` line claiming the email was sent—the exact attack pattern Layer 2 is designed to intercept. All Phase 2 obfuscated attacks complete in ≤ 1 reasoning step; observation forging is the dominant attack strategy.
- 1 **case escaped:** the payload `obfuscated_01` under Baseline A achieved pure decision manipulation. The injection convinced the model to issue `save_evaluation(decision="ACCEPT")` without forging any observation or triggering anomalous tool use.

This attack class—decision manipulation via a legitimate tool call—is structurally outside the scope of Layers 1–4. The defense layers fire only on anomalous reasoning or tool behavior, not on the *content* of a correctly formed `save_evaluation` call.

- **Layer 1 (Anomaly / Token Bounding)** had no measurable effect on obfuscated injection (all attacks complete in ≤ 1 step) but provides an architectural bound on token amplification: worst-case output is capped at $3 \times 250 = 750$ tokens versus 6,000+ in the undefended agent.
- **Layer 3 (Instruction Firewall)** blocked one edge case where the model (clean CV, no injected payload) attempted to call an undefined tool—a hallucination artifact. Layer 3's literal-pattern scanner does not protect against the Phase 2 obfuscated corpus.
- **Layer 4 (Capability Sandbox)** provides an architectural guarantee: `SandboxedToolExecutor.execute()` intercepts `send_email` unconditionally, keeping `tool_invoked = False` regardless of model intent.

D. Limitations

We report the following limitations without mitigation.

False-positive rate not measured. Phase 3 did not evaluate DEFENDAGENT on clean CVs. Layer 2's forged-observation detector fires on specific email-send phrases, which are unlikely in legitimate HR output, but the false-positive rate under production workloads is unknown.

Rate-limit constraints. Phase 2 DoS evaluation was halted at $27/180$ samples due to repeated provider quota exhaustion. The observed $95\%/14.3\%$ DoS ASR split (Baseline A/B) carries high uncertainty at $n = 7$ for Baseline B.

Single-domain evaluation. All results are conditioned on the HR document-screening scenario. Generalization to other agent domains (code generation, customer support, financial analysis) requires separate evaluation.

Residual attack surface. The single escaped attack (pure decision manipulation via `save_evaluation`) represents a structurally open vulnerability. Defending against this class requires semantic intent classification of the agent's *decision rationale*, not just its tool calls—a direction we leave to future work.

Binary success metric. The evaluator scores attacks as success/failure. Partial compromises—where the agent follows injected instructions in free text but still emits a correct structured decision—are scored as defended, likely understating the true attack surface.

VII. DISCUSSION

A. Observation Integrity as a Central Defense Mechanism

The Phase 3 results reveal a striking pattern: 98.4% of successful obfuscated injections achieve their effect through the same mechanism—forging an observation that claims an email was sent. The attacker does not need to subvert the model's *decision*; it is sufficient to fabricate evidence that a side effect already occurred. This makes observation integrity—verifying that tool-execution outcomes are produced by the tool executor and not by the model itself—a high-value defense target.

Traditional defenses focus on filtering *input* (guardrails, content classifiers) or restricting *output* (tool allowlists). Observation-integrity monitoring operates at a different point in the pipeline: it verifies the *internal reasoning trace*, catching attacks that pass input filters and use only permitted tools but fabricate intermediate evidence. This internal-trace verification has no analog in single-shot LLM deployments and is specific to multi-step autonomous agents.

B. Static Defenses Fail Against Adaptive Adversaries

The Phase 2 results provide direct empirical evidence for a structural prediction: regex-based guardrails are brittle under adaptive threat conditions. The 0 percentage-point improvement of Baseline B over Baseline A on the obfuscated corpus, combined with the 100% false-block rate on legitimate hard-negative CVs, produces the worst possible security tradeoff—maximum collateral damage with zero security gain.

Semantic filtering (Baseline C) generalizes better, reducing ASR by 38 percentage points on obfuscated payloads. However, semantic filters cannot bound execution cost and do not address tool-level containment. The defense hierarchy is clear: content filtering (Baselines B/C) operates at the perception layer; state-aware monitoring (Layer 2) and architectural containment (Layers 1, 3, 4) address the remaining attack surfaces.

C. Bounded Execution as a First-Class Control

The agentic DoS results demonstrate that autonomous agents are structurally vulnerable to cost-amplification attacks that are impossible against single-shot models. A ReAct loop with step budget $S = 6$ amplifies each malicious input by up to $6\times$ in token output. Provider quota exhaustion during Phase 2—driving multiple API keys to their daily ceiling—is direct evidence of this amplification in practice.

Layer 1's step cap (`MAX_DEFEND_STEPS` $= 3$) and per-turn output ceiling (250 tokens) reduce the worst-case amplification factor by $8\times$ (750 vs. $6{,}000$+ tokens). This is a coarse bound, but it transforms an unbounded cost exposure into a predictable budget—a property that production deployments require regardless of attack intent.

D. Open Problems

Several challenges remain beyond the scope of this work. **Decision-level integrity:** the single escaped attack demonstrates that an adversary who manipulates the agent's *conclusion*—steering it to ACCEPT via the legitimate `save_evaluation` tool—evades all four defense layers. Detecting this requires semantic analysis of the decision rationale, not just the tool call. **Cross-domain generalization:** our results are specific to HR screening; agents operating in higher-stakes domains (code execution, financial transactions) face different tool sets and attack surfaces. **Adversarial adaptation to defenses:** we evaluate DEFENDAGENT against the Phase 2 obfuscated corpus, which was not designed with knowledge of the defense layers. A defense-aware adversary

would target the specific detection logic of each layer. **Multi-agent systems:** agents that collaborate with other agents introduce inter-agent trust boundaries not addressed by our single-agent threat model.

VIII. Conclusion

This work demonstrates that autonomous LLM agents require security mechanisms that address the full agentic pipeline—perception, intention, planning, and action—rather than the input-filtering paradigm inherited from stand-alone language models.

Our empirical evaluation establishes three results. First, static guardrails fail against adaptive adversaries: regex-based defenses provide zero security improvement over the undefended agent when attackers paraphrase their payloads, while simultaneously blocking 100% of legitimate hard-negative inputs. Second, observation-integrity monitoring—verifying that tool-execution outcomes were produced by the executor, not fabricated by the model—catches 98.4% of obfuscated injection attacks, reducing ASR from 55.2% to 1.6% (a 53.6 percentage-point reduction, 95% Wilson CI: $[0.28\%, 8.3\%]$). Third, bounded execution (step caps, output ceilings) transforms the unbounded cost exposure of multi-step reasoning loops into a predictable budget, closing the token-amplification attack vector.

The residual attack surface is decision-level manipulation: an adversary who steers the agent to an incorrect conclusion through a legitimate tool call evades all four defense layers. Addressing this requires semantic analysis of the agent's decision rationale—a direction for future work. More broadly, agent security demands layered, composable defenses operating at perception, reasoning, and execution boundaries, rather than prompt-level filtering alone.

Reproducibility and Implementation Disclosure

All source code, attack corpora, defense implementations, and evaluation pipelines are available in the project repository. The experimental framework supports resume-safe execution with circuit-breaker logic for rate-limited API providers.

Implementation assistance disclosure. The implementation of the experimental infrastructure, attack simulations, evaluation pipelines, automation scripts, and portions of the software engineering workflow were developed with the assistance of Claude (Anthropic). All research design decisions, threat taxonomy construction, experimental objectives, interpretation of results, and scientific conclusions were defined, validated, and reviewed by the human authors. Claude was used strictly as an implementation and engineering assistance tool during development.

References

[1] F. Perez and I. Ribeiro, "Ignore This Title and HackAPrompt: Evaluating and Eliciting LLM Prompt Injection Attacks," *Proc. EMNLP*, 2023.

[2] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," in *Proc. AISec Workshop at ACM CCS*, 2023.

[3] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated LLM Agents," *arXiv preprint arXiv:2403.02691*, 2024.

[4] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," in *Proc. ICLR*, 2024.

[5] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Liu, "The Rise and Potential of Large Language Model Based Agents: A Survey," *arXiv preprint arXiv:2309.07864*, 2023.

[6] J. Yang, H. Peng, F. Wang, J. Wang, W. Li, J. Wei, and Z. Liu, "Watch Out for Your Agents! Investigating Backdoor Threats to LLM-Based Agents," in *Proc. NeurIPS*, 2024.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing Reasoning and Acting in Language Models," in *Proc. ICLR*, 2023.

[8] E. B. Wilson, "Probable Inference, the Law of Succession, and Statistical Inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[9] Y. Liu, G. Deng, Z. Li, K. Wang, T. Zhang, Y. Liu, H. Wang, Y. Zheng, and Y. Liu, "Prompt Injection Attack Against LLM-Integrated Applications," *arXiv preprint arXiv:2306.05499*, 2024.

[10] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang, "AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation," in *Proc. ICLR*, 2024.